
Practical : 1	Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort
Date : 01/07/2026	

Aim:

To implement and analyze different sorting algorithms such as **Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort.**

Objective :- To implement five major sorting algorithms (Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quicksort) in C++ using arrays, and to evaluate their combined time and space complexity characteristics.

SORTING ALGORITHMS

1. Bubble Sort

1. Start.
2. Read the array and size n.
3. Compare adjacent elements.
4. If the first element is greater than the next element, swap them.
5. Repeat the comparison for the remaining elements.
6. After each pass, the largest element moves to the end.
7. Repeat until the array is sorted.
8. Stop.

2. Selection Sort

1. Start.
2. Read the array and size n.
3. Select the first unsorted position.
4. Find the minimum element in the remaining array.
5. Swap the minimum element with the current position.
6. Move to the next position.
7. Repeat until the array is sorted.
8. Stop.

3. Insertion Sort

1. Start.
2. Read the array and size n.
3. Select the next element as key.
4. Compare the key with elements in the sorted portion.
5. Shift larger elements one position to the right.
6. Insert the key in its correct position.
7. Repeat for all elements.
8. Stop.

4. Merge Sort

1. Start.
2. Divide the array into two halves.
3. Recursively sort the left half.
4. Recursively sort the right half.
5. Merge the two sorted halves.
6. Repeat until the complete array is sorted.
7. Stop.

5. Quick Sort

1. Start.
2. Select the last element as the pivot.
3. Partition the array around the pivot.
4. Place smaller elements before the pivot.
5. Place larger elements after the pivot.
6. Recursively sort the left and right portions.
7. Stop.

Program :-

```
#include <iostream>

using namespace std;

// Function to print array elements

void printArray(int arr[], int size) {

    for (int i = 0; i < size; i++) {
```

```
cout << arr[i] << " ";

}

cout << endl;

}

// 1. Bubble Sort

void bubbleSort(int arr[], int n) {

for (int i = 0; i < n - 1; i++) {

for (int j = 0; j < n - i - 1; j++) {

if (arr[j] > arr[j + 1]) {

int temp = arr[j];

arr[j] = arr[j + 1];

arr[j + 1] = temp;

}

}

}

}

// 2. Selection Sort

void selectionSort(int arr[], int n) {

for (int i = 0; i < n - 1; i++) {

int minIndex = i;

for (int j = i + 1; j < n; j++) {

if (arr[j] < arr[minIndex]) {

minIndex = j;
```

```
}  
  
}  
  
int temp = arr[minIndex];  
  
arr[minIndex] = arr[i];  
  
arr[i] = temp;  
  
}  
  
}  
  
// 3. Insertion Sort  
  
void insertionSort(int arr[], int n) {  
    for (int i = 1; i < n; i++) {  
        int key = arr[i];  
        int j = i - 1;  
        while (j >= 0 && arr[j] > key) {  
            arr[j + 1] = arr[j];  
            j--;  
        }  
        arr[j + 1] = key;  
    }  
}  
  
// Merge helper function  
  
void merge(int arr[], int left, int mid, int right) {  
    int n1 = mid - left + 1;  
    int n2 = right - mid;
```

```
int L[100], R[100];

for (int i = 0; i < n1; i++)

L[i] = arr[left + i];

for (int j = 0; j < n2; j++)

R[j] = arr[mid + 1 + j];

int i = 0, j = 0, k = left;

while (i < n1 && j < n2) {

if (L[i] <= R[j]) {

arr[k] = L[i];

i++;

} else {

arr[k] = R[j];

j++;

}

k++;

}

while (i < n1) {

arr[k] = L[i];

i++;

k++;

}

while (j < n2) {

arr[k] = R[j];
```

```
j++;  
  
k++;  
  
}  
  
}  
  
// 4. Merge Sort  
  
void mergeSort(int arr[], int left, int right) {  
  
    if (left < right) {  
  
        int mid = left + (right - left) / 2;  
  
        mergeSort(arr, left, mid);  
  
        mergeSort(arr, mid + 1, right);  
  
        merge(arr, left, mid, right);  
  
    }  
  
}  
  
// Partition function for Quick Sort  
  
int partition(int arr[], int low, int high) {  
  
    int pivot = arr[high];  
  
    int i = low - 1;  
  
    for (int j = low; j < high; j++) {  
  
        if (arr[j] < pivot) {  
  
            i++;  
  
            int temp = arr[i];  
  
            arr[i] = arr[j];  
  
            arr[j] = temp;
```

```

    }

    }

    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return i + 1;
}

// 5. Quick Sort

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int originalArr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(originalArr) / sizeof(originalArr[0]);
    int arr[100];

    cout << "Original Array: ";

    printArray(originalArr, n);

```

```

// Bubble Sort

```

```
for (int i = 0; i < n; i++)

arr[i] = originalArr[i];

bubbleSort(arr, n);

cout << "\n1. Bubble Sort Result: ";

printArray(arr, n);

// Selection Sort

for (int i = 0; i < n; i++)

arr[i] = originalArr[i];

selectionSort(arr, n);

cout << "2. Selection Sort Result: ";

printArray(arr, n);

// Insertion Sort

for (int i = 0; i < n; i++)

arr[i] = originalArr[i];

insertionSort(arr, n);

cout << "3. Insertion Sort Result: ";

printArray(arr, n);

// Merge Sort

for (int i = 0; i < n; i++)

arr[i] = originalArr[i];

mergeSort(arr, 0, n - 1);

cout << "4. Merge Sort Result: ";

printArray(arr, n);
```

```
// Quick Sort

for (int i = 0; i < n; i++)

arr[i] = originalArr[i];

quickSort(arr, 0, n - 1);

cout << "5. Quicksort Result: ";

printArray(arr, n);

return 0;

}
```

Output:

```
Output
Original Array: 64 34 25 12 22 11 90

1. Bubble Sort Result: 11 12 22 25 34 64 90
2. Selection Sort Result: 11 12 22 25 34 64 90
3. Insertion Sort Result: 11 12 22 25 34 64 90
4. Merge Sort Result: 11 12 22 25 34 64 90
5. Quicksort Result: 11 12 22 25 34 64 90

=== Code Execution Successful ===
```

Analysis:-

The algorithms tested can be naturally grouped into two categories based on their design patterns: **Quadratic-time basic sorts** $O(n^2)$ and **Logarithmic-time advanced divide-and-conquer sorts** $O(n \log n)$

Time Complexity Summary

Bubble, Selection, and Insertion Sort utilize nested loop structures. In average and worst-case setups, they complete roughly n^2 computational comparison loops, giving them an $O(n^2)$ quadratic rate of growth. Insertion and optimized Bubble Sort perform exceptionally well $O(n)$ if the dataset is already mostly sorted.

Merge Sort and Quicksort split the data into smaller fractions continuously. This divide-and-conquer strategy dramatically cuts down operations to an **$O(n \log n)$** average rate.

Quicksort can degrade to **$O(n^2)$** if the pivot choices consistently isolate single elements

Instead of balanced splits

Sorting Algorithms	Time Complexity			Space Complexity
	Best Case	Average Case	Worst Case	Worst Case
Bubble Sort	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$
Selection Sort	$O(N^2)$	$O(N^2)$	$O(N^2)$	$O(1)$
Insertion Sort	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$
Quick Sort	$O(N \log N)$	$O(N \log N)$	$O(N^2)$	$O(N)$
Merge Sort	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(N)$

Space Complexity Summary

- a) **Bubble, Selection, and Insertion Sort** require **$O(1)$** space because they arrange elements completely in-place without generating extra lists.
- b) **Merge Sort** requires a linear auxiliary footprint of **$O(n)$** memory to construct temporary arrays during active array blending operations.
- c) **Quicksort** uses an execution stack footprint of **$O(\log n)$** on average to keep track of recursive partition boundaries.

Conclusion :-

This practical highlights the direct real-world trade-off between algorithmic simplicity and scalability:

For Small Datasets: Simple algorithms like **Insertion Sort** and **Bubble Sort** are optimal because they require negligible implementation complexity and execute completely in place ($O(1)$ auxiliary space).

For Large Datasets: Divide-and-conquer algorithms like **Merge Sort** and **Quicksort** are vastly superior because their $O(n \log n)$ runtime allows them to process massive files cleanly. **Merge Sort** guarantees reliable stability but requires a higher memory profile ($O(n)$), while **Quicksort** offers highly rapid in-place processing under typical runtime conditions.

Practical : 2	Implementation and Time analysis of linear and binary search algorithm.
Date : 08/07/2026	

Aim: To implement and analyze **Linear Search and Binary Search** algorithms.

A) LINEAR SEARCH :-

Objective :- To implement the Linear Search algorithm in C++ using an array, and to analyze its performance across best, worst, and average-case scenarios.

Linear Search Algorithm

1. Start.
2. Read the array and target element.
3. Start from the first element.
4. Compare each element with the target.
5. If the element matches, return its index.
6. Otherwise, continue to the next element.
7. If the end is reached, return -1.
8. Display the result.
9. Stop.

Program :-

```
#include <iostream>

using namespace std;

// Function for Linear Search

int linearSearch(int arr[], int size, int target) {

    for (int i = 0; i < size; i++) {

        if (arr[i] == target) {

            return i;

        }

    }

    return -1;

}
```

```
    }  
  
    }  
  
    return -1;  
  
    }  
  
int main()  
  
int arr[] = {12, 45, 7, 23, 56, 90, 3};  
  
int size = sizeof(arr) / sizeof(arr[0]);  
  
int target;  
  
cout << "Enter the number you want to search for: ";  
  
cin >> target;  
  
int result = linearSearch(arr, size, target);  
  
if (result != -1) {  
    cout << "Element found at index: " << result << endl;  
    }  
    else {  
        cout << "Element not found in the array." << endl;  
    }  
  
return 0;  
  
}
```

Output:

```
Output
Enter the number you want to search for: 23
Element found at index: 3

=== Code Execution Successful ===
```

Analysis:-

1. Time Complexity Analysis

The execution time of Linear Search relies on the number of comparisons made within the forloop relative to the array size **n**.

- **Best Case: $O(1)$**

This happens when the target element matches the very first element of the array (**arr[0]**). The loop terminates instantly on its first pass, executing exactly 1 comparison.

- **Worst Case: $O(n)$**

This happens if the target element is located at the very end of the array (**arr[size-1]**) or is completely missing from it. The loop must iterate over every single item, executing exactly **n** comparisons.

- **Average Case: $O(n)$**

Assuming the target element has an equal probability of being located at any index, the algorithm will inspect roughly half the array ($\frac{n+1}{2}$ elements) before finding it. Dropping constants leaves a linear growth rate.

2. Space Complexity Analysis

- **Space Complexity: $O(1)$**

The algorithm performs its operations completely in place inside the existing array. The only extra memory allocated during execution is for a few basic variables (i and size). Because this memory allocation does not grow with the array size, it requires constant auxiliary space.

Conclusion :-

Through this practical, we observed that Linear Search is a simple and straightforward searching technique. Its primary benefit is flexibility—it does not care whether the input data is sorted or unsorted. However, its time complexity scales linearly $O(n)$, meaning performance drops dramatically as the size of the dataset expands, making it inefficient for large arrays.

B)BINARY SEARCH :-

Objective :- To implement the Binary Search algorithm in C++ using a sorted array, and to analyze its performance across best, worst, and average-case scenarios.

Binary Search Algorithm

Condition: The array must be sorted.

1. Start.
2. Read the sorted array and target element.
3. Set `low = 0`.
4. Set `high = n - 1`.
5. Calculate `mid`.
6. Compare `arr[mid]` with the target.
7. If equal, return the index.
8. If `arr[mid] < target`, search the right half.
9. Otherwise, search the left half.
10. Repeat until the element is found or `low > high`.
11. If not found, return -1.
12. Stop.

Program:

```
#include <iostream>
using namespace std;

// Function for Binary Search
// Requires sorted array
int binarySearch(int arr[], int size, int target) {

    int low = 0;
    int high = size - 1;

    while (low <= high) {

        int mid = low + (high - low) / 2;

        if (arr[mid] == target) {
```

```
        return mid;
    }
    else if (arr[mid] < target) {
        low = mid + 1;
    }
    else {
        high = mid - 1;
    }
}

return -1;
}

int main() {

    // Array must be sorted
    int arr[] = {3, 7, 12, 23, 45, 56, 90};

    int size = sizeof(arr) / sizeof(arr[0]);

    int target;

    cout << "Enter the number you want to search for: ";
    cin >> target;

    int result = binarySearch(arr, size, target);

    if (result != -1) {
        cout << "Element found at index: " << result << endl;
    }
    else {
        cout << "Element not found in the array." << endl;
    }

    return 0;
}
```

Output:

```
Output
Enter the number you want to search for: 7
Element found at index: 2

=== Code Execution Successful ===
```

Analysis:-

A. Time Complexity Analysis

The execution time of Binary Search is determined by how many times the `while` loop runs to split the sorted array in half to pinpoint the target value.

- **Best Case: $O(1)$**

This occurs when the target value happens to be exactly at the middle position (`arr[mid]`) on the very first calculation. The loop matches immediately and terminates after just 1 comparison.

- **Worst Case: $O(\log n)$**

This occurs if the target element is at the extreme far ends of the array or is completely missing. Because the algorithm continuously divides the remaining search zone into halves with every step ($n/2, n/4$), the total number of operations grows logarithmically with respect to the array size n .

- **Average Case: $O(\log n)$**

On average, most successful lookups or missed searches will find their answer deeper within the binary division tree structure, yielding an overall logarithmic growth rate.

B. Space Complexity Analysis

- **Space Complexity: $O(1)$**

This iterative implementation utilizes constant auxiliary space. It updates a

fixed set of tracking boundaries (`low`, `high`, `mid`) directly within the local memory stack. Since no dynamic arrays are created and no recursive functional call frames stack up, memory consumption never changes regardless of how large n becomes.

Conclusion:-

Through this practical, we observed that Binary Search is an exceptionally fast searching mechanism that runs with a logarithmic time efficiency (**$O(\log n)$**). This makes it highly optimal for scaling across massive datasets where linear scanning becomes far too slow. The only trade-off is its rigid prerequisite constraint: the dataset **must be strictly pre-sorted** for the split-and-discard logic to work.

Practical : 3	Implementation of max-heap sort algorithm
Date : 05/08/2026	

Aim: To implement and analyze the **Heap Sort algorithm using a Max-Heap**.

Objective :- To implement the Heap Sort algorithm using a Max-Heap structure in C++, and to evaluate its time and space complexities.

MAX-HEAP SORT Algorithm

1. Start.
2. Read the array and size n.
3. Build a Max-Heap.
4. Start from the last non-leaf node.
5. Compare the node with its left and right children.
6. Find the largest element.
7. If required, swap the node with the largest child.
8. Apply heapify recursively.
9. Swap the root element with the last element.
10. Reduce the heap size.
11. Apply heapify again.
12. Repeat until all elements are sorted.
13. Display the sorted array.
14. Stop.

Program:

```
#include <iostream>
using namespace std;

// Function to print array elements
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }

    cout << endl;
}

// Function to heapify a subtree
```

```
void heapify(int arr[], int n, int i) {  
  
    int largest = i;  
  
    int left = 2 * i + 1;  
    int right = 2 * i + 2;  
  
    // If left child is larger than root  
    if (left < n && arr[left] > arr[largest]) {  
        largest = left;  
    }  
  
    // If right child is larger  
    if (right < n && arr[right] > arr[largest]) {  
        largest = right;  
    }  
  
    // If largest is not root  
    if (largest != i) {  
  
        int temp = arr[i];  
        arr[i] = arr[largest];  
        arr[largest] = temp;  
  
        // Heapify affected subtree  
        heapify(arr, n, largest);  
    }  
}  
  
// Function to perform Heap Sort  
void heapSort(int arr[], int n) {  
  
    // Step 1: Build Max-Heap  
    for (int i = n / 2 - 1; i >= 0; i--) {  
        heapify(arr, n, i);  
    }  
  
    // Step 2: Extract elements
```

```
for (int i = n - 1; i > 0; i--) {  
  
    // Move root to the end  
    int temp = arr[0];  
    arr[0] = arr[i];  
    arr[i] = temp;  
  
    // Heapify reduced heap  
    heapify(arr, i, 0);  
    }  
    }  
  
int main() {  
  
int arr[] = {45, 12, 85, 32, 7, 90, 60};  
  
int n = sizeof(arr) / sizeof(arr[0]);  
  
cout << "Original Array: ";  
    printArray(arr, n);  
  
    heapSort(arr, n);  
  
cout << "Sorted Array (Heap Sort): ";  
    printArray(arr, n);  
  
    return 0;  
    }
```

Output:

```
Output  
Original Array: 45 12 85 32 7 90 60  
Sorted Array (Heap Sort): 7 12 32 45 60 85 90  
  
=== Code Execution Successful ===
```

Complexity Analysis:-

A. Time Complexity Analysis

Heap Sort operates in two distinct phases: building the initial max-heap and repeatedly extracting the maximum element.

- **Building the Max-Heap:** Converting an unsorted array of size n into a valid max-heap takes $O(n)$ time.
- **Heapify during Extraction:** Extracting the root element and pushing it to the back requires running the heapify function. Since a binary heap has a max height of $\log n$, each extraction swap costs $O(\log n)$ time. Doing this for all n elements takes $O(n \log n)$ time.

Combining these two phases gives Heap Sort a remarkably consistent time profile:

- **Best Case:** $O(n \log n)$
- **Average Case:** $O(n \log n)$
- **Worst Case:** $O(n \log n)$

B. Space Complexity Analysis

- **Space Complexity:** $O(1)$ (Constant Space)

The sorting process happens completely in-place. The elements are rearranged within the original input array itself, requiring no external lists or auxiliary memory allocation.

Conclusion :-

Through this practical, we successfully implemented Heap Sort using a Max-Heap structure. The primary advantage of Heap Sort is its absolute dependability—it guarantees a highly efficient $O(n \log n)$ time complexity across all input conditions (best, worst, and average cases) while matching the in-place $O(1)$ space efficiency of simpler quadratic sorts.

Practical : 4	Implementation and Time analysis of factorial program using iterative and recursive method
Date : 15/07/2026	

Aim: To implement and compare **iterative and recursive methods for calculating the factorial number.**

Objective :-To implement programs to calculate the factorial of a given number in C++ using both iterative and recursive methods, and to analyze and compare their respective time and space complexities.

A. Iterative Factorial Algorithm

1. Start.
2. Read n.
3. Initialize result = 1.
4. Run a loop from 1 to n.
5. Multiply result by the current value.
6. Return the result.
7. Stop.

B. Recursive Factorial Algorithm

1. Start.
2. Read n.
3. If $n \leq 1$, return 1.
4. Otherwise calculate $n \times \text{factorial}(n-1)$.
5. Continue until the base case is reached.
6. Return the result.
7. Stop.

Program:

```
#include <iostream>

using namespace std;

// 1. Iterative Method

long long factorialIterative(int n) {
```

```
        long long result = 1;

        for (int i = 1; i <= n; i++) {

            result *= i;

        }

        return result;

    }

// 2. Recursive Method

long long factorialRecursive(int n) {

    // Base Case

    if (n <= 1) {

        return 1;

    }

    // Recursive Case

    return n * factorialRecursive(n - 1);

}

int main() {

    int num;

    cout << "Enter a positive integer: ";

    cin >> num;
```

```
        if (num < 0) {  
            cout << "Factorial of a negative number is not defined."  
                << endl;  
        }  
        else {  
            // Iterative Method  
            long long iterativeResult =  
                factorialIterative(num);  
            cout << "\n--- Iterative Method ---" << endl;  
            cout << "Factorial of " << num  
                << " is: " << iterativeResult << endl;  
            // Recursive Method  
            long long recursiveResult =  
                factorialRecursive(num);  
            cout << "\n--- Recursive Method ---" << endl;  
            cout << "Factorial of " << num  
                << " is: " << recursiveResult << endl;  
        }  
        return 0;  
    }  
}
```

Output:

```
Output

Enter a positive integer: 10

--- Iterative Method ---
Factorial of 10 is: 3628800

--- Recursive Method ---
Factorial of 10 is: 3628800

=== Code Execution Successful ===
```

Analysis:

Time Complexity Analysis

Let n be the input number for which we need to calculate the factorial.

Iterative Method:

- The iterative function executes a simple for loop that runs exactly n times to multiply the values from **1 to n** . Since there are no nested steps, the execution time grows linearly with the input size.

Time Complexity = $O(n)$

Recursive Method:

- The recursive function calls itself continuously, decremented by 1 at each step:
- **$T(n) = T(n - 1) + O(1)$**
- This sequence continues until it hits the **base case ($n \leq 1$)**. This creates a total chain of n recursive function calls, resulting in a linear growth rate.

Time Complexity = $O(n)$

Space Complexity Analysis

Iterative Method:

- The loop uses only a few fixed-size scalar variables (result, i) to run. The amount of memory occupied stays completely constant and independent of the input value n.

Space Complexity = $O(1)$

Recursive Method:

- Every time a function calls itself, a new stack frame is pushed onto the system call stack to store the current state and return address. For an input of size n, the system must hold n active stack frames simultaneously before hitting the base case and unwinding.

Space Complexity = $O(n)$

Conclusion :-

While both methods successfully calculate the factorial of a number in linear time $O(n)$, their system overhead profiles are vastly different:

1. The **Iterative Method** is highly memory efficient. It works with a constant auxiliary footprint of $O(1)$ space, making it safe from stack exhaustion limits.
2. The **Recursive Method** is mathematically expressive and maps directly to the recursive definition of factorials. However, it requires linear auxiliary memory space of $O(n)$ due to the call stack overhead. For large inputs, this makes recursion prone to a **stack overflow** crash.